

Krishnamargam - Deployment Guide

Version: 1.0
Date: May 2026
Stack: Next.js 14 + Supabase + Vercel + Razorpay + Resend + MSG91
Maintained by: BabyStepsIndia.in

Overview

This guide takes you from zero to a live production deployment. Follow each phase in order.
Estimated total time: **3-4 hours** for a first-time setup.

Phase	What you'll do	Time
1	Create all third-party accounts	30 min
2	Set up Supabase (database + auth)	45 min
3	Set up the Next.js project locally	30 min
4	Configure all integrations	30 min
5	Seed the database	15 min
6	Deploy to Vercel	20 min
7	Post-deployment verification	30 min

Phase 1 — Create Third-Party Accounts

1.1 Supabase

- Go to supabase.com → Sign up with GitHub
- Click **New Project**
- Set:
 - Name:** `krishnamargam`
 - Database Password:** generate a strong password — save it securely
 - Region:** `Southeast Asia (Singapore)` — closest to Indian users

4. Wait ~2 minutes for the project to provision
5. Go to **Settings** → **API** and copy:
 - `Project URL` → this is your `NEXT_PUBLIC_SUPABASE_URL`
 - `anon public` key → this is your `NEXT_PUBLIC_SUPABASE_ANON_KEY`
 - `service_role` key → this is your `SUPABASE_SERVICE_ROLE_KEY` ⚠️ keep this secret

1.2 Razorpay

1. Go to razorpay.com → Sign up
2. Complete KYC (required for live payments — use test mode initially)
3. Go to **Settings** → **API Keys** → Generate Test Keys
4. Copy:
 - `Key ID` → `RAZORPAY_KEY_ID`
 - `Key Secret` → `RAZORPAY_KEY_SECRET`
5. Go to **Settings** → **Webhooks** → Add Webhook
 - URL: `https://krishnamargam.babystepsindia.in/api/payment/webhook`
 - Events: tick `payment.captured` and `payment.failed`
 - Copy the **Webhook Secret** → `RAZORPAY_WEBHOOK_SECRET`

1.3 Resend (Email)

1. Go to resend.com → Sign up
2. Go to **API Keys** → Create API Key → copy it → `RESEND_API_KEY`
3. Go to **Domains** → Add Domain → `babystepsindia.in`
4. Add the DNS records shown (DKIM, SPF, DMARC) to your domain registrar
5. Wait for domain verification (5-30 minutes)
6. Set `RESEND_FROM_EMAIL=noreply@babystepsindia.in`

1.4 MSG91 (WhatsApp OTP)

1. Go to msg91.com → Sign up
2. Go to **WhatsApp** → Apply for WhatsApp Business API
3. Once approved, go to **API** → copy:
 - Auth Key → `MSG91_AUTH_KEY`
4. Go to **WhatsApp** → **Templates** → Create a template for OTP:
 - Template name: `krishnamargam_otp`

- Body: `Your Krishnamargam verification code is {{1}}. Valid for 10 minutes.`
- Copy the Template ID → `MSG91_TEMPLATE_ID`

5. Copy your Sender ID → `MSG91_SENDER_ID`

Note: WhatsApp Business API approval takes 1-3 business days. During development, use SMS OTP as fallback.

1.5 Vercel

1. Go to vercel.com → Sign up with GitHub
 2. No configuration needed yet — we'll connect the project in Phase 6
-

Phase 2 — Set Up Supabase

2.1 Run the Database Schema

1. In your Supabase project, go to **SQL Editor**
2. Click **New Query**
3. Copy the entire SQL from the **Database Schema document** (all sections: extensions, tables, triggers, RLS policies, views)
4. Paste and click **Run**
5. You should see: `Success. No rows returned`

If you see errors, run each section separately to identify the issue.

2.2 Configure Supabase Auth

Go to **Authentication** → **Settings**:

Email settings:

- Enable email confirmations: ☒ ON
- Confirm email: ☒ ON
- Secure email change: ☒ ON
- Email OTP expiry: `86400` (24 hours)

Site URL:

`https://krishnamargam.babystepsindia.in`

```
https://krishnamargam.babystepsindia.in/auth/callback
https://krishnamargam.babystepsindia.in/login
http://localhost:3000/auth/callback
http://localhost:3000/login
```

2.3 Configure Email Templates

Go to **Authentication** → **Email Templates**:

Confirm signup template:

```
html

<h2>Welcome to Krishnamargam 🙏</h2>
<p>Please confirm your email address to begin your spiritual journey.</p>
<a href="{{ .ConfirmationURL }}">Confirm my email</a>
<p>This link expires in 24 hours.</p>
<p>© 2026 Krishnamargam - BabyStepsIndia.in</p>
```

Reset password template:

```
html

<h2>Reset your Krishnamargam password</h2>
<p>Click the link below to reset your password. This link expires in 30 minutes.</p>
<a href="{{ .ConfirmationURL }}">Reset my password</a>
<p>If you didn't request this, you can safely ignore this email.</p>
<p>© 2026 Krishnamargam - BabyStepsIndia.in</p>
```

2.4 Set Up Google OAuth (Optional — Phase 5)

Go to **Authentication** → **Providers** → **Google**:

1. Create a project in Google Cloud Console
2. Enable Google+ API
3. Create OAuth 2.0 credentials:
 - Authorised origins: `https://krishnamargam.babystepsindia.in`
 - Authorised redirect URI: `https://<your-supabase-project>.supabase.co/auth/v1/callback`
4. Copy Client ID and Secret into Supabase Google provider settings

2.5 Configure Storage (Optional)

If you later add profile photos or course thumbnails:

1. Go to **Storage** → Create bucket named `avatars`
2. Set to **Public**
3. Add RLS policy: authenticated users can upload to their own folder

2.6 Get the Database Connection String

Go to **Settings** → **Database** → **Connection string**:

- Copy the **URI** format string
 - Replace `[YOUR-PASSWORD]` with your database password
 - Save as `DATABASE_URL` (needed for migrations)
-

Phase 3 — Set Up the Next.js Project

3.1 Create the Project

```
bash

npx create-next-app@14 krishnamargam \
  --typescript \
  --tailwind \
  --eslint \
  --app \
  --src-dir \
  --import-alias "@/*"

cd krishnamargam
```

3.2 Install Dependencies

```
bash
```

```
# Supabase
npm install @supabase/supabase-js @supabase/ssr

# Razorpay
npm install razorpay

# Resend
npm install resend

# Forms and validation
npm install zod react-hook-form @hookform/resolvers

# Charts (for analytics)
npm install recharts

# CSV export
npm install papaparse
npm install --save-dev @types/papaparse

# Date handling
npm install date-fns

# UI utilities
npm install clsx tailwind-merge lucide-react

# Telugu font
# Add to app/layout.tsx via next/font/google
```

3.3 Project Structure

```
krishnamargam/
├─ src/
│   └─ app/
│       └─ (student)/           # Student-facing pages
│           └─ page.tsx        # Home
│           └─ explore/
│           └─ course/[slug]/
│           └─ watch/[slug]/
│           └─ my-courses/
│           └─ enroll/[courseId]/
│           └─ login/
│           └─ register/
```

```
| | | └─ forgot-password/
| | └─ (admin)/          # Admin pages
| |   └─ admin/
| |     └─ login/
| |     └─ paths/
| |     └─ courses/
| |     └─ enrollments/
| |     └─ qa/
| |     └─ analytics/
| |     └─ exports/
| | └─ api/              # API routes
| |   └─ courses/
| |   └─ paths/
| |   └─ auth/
| |     └─ otp/
| |   └─ enroll/
| |   └─ watch/
| |   └─ notes/
| |   └─ progress/
| |   └─ qa/
| |   └─ quiz/
| |   └─ payment/
| |     └─ create-order/
| |     └─ webhook/
| |   └─ admin/
| |     └─ paths/
| |     └─ courses/
| |     └─ enrollments/
| |     └─ qa/
| |     └─ analytics/
| |     └─ exports/
| | └─ auth/
| |   └─ callback/      # Supabase auth callback
| | └─ layout.tsx
| └─ components/
|   └─ ui/              # Shared UI components
|   └─ student/         # Student-facing components
|   └─ admin/           # Admin components
| └─ lib/
|   └─ supabase/
|     └─ client.ts      # Browser client
|     └─ server.ts      # Server client
|     └─ middleware.ts  # Auth middleware
| └─ razorpay.ts
```

```

|   |   └─ resend.ts
|   |   └─ msg91.ts
|   |   └─ utils.ts
|   └─ types/
|       └─ database.ts          # Generated Supabase types
|   └─ middleware.ts          # Next.js middleware
└─ public/
└─ .env.local
└─ .env.example
└─ next.config.js

```

3.4 Create Environment File

Create `.env.local` in the project root:

```

env

# Supabase
NEXT_PUBLIC_SUPABASE_URL=https://xxxxxxxxxxxxx.supabase.co
NEXT_PUBLIC_SUPABASE_ANON_KEY=eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9...
SUPABASE_SERVICE_ROLE_KEY=eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9...

# Razorpay
RAZORPAY_KEY_ID=rzp_test_xxxxxxxxxxxxxxxxxx
RAZORPAY_KEY_SECRET=xxxxxxxxxxxxxxxxxxxxxxxxxxxx
RAZORPAY_WEBHOOK_SECRET=xxxxxxxxxxxxxxxxxxxxxxxxxxxx

# Resend
RESEND_API_KEY=re_xxxxxxxxxxxxxxxxxxxxxxxxxxxxx
RESEND_FROM_EMAIL=noreply@babystepsindia.in

# MSG91 (WhatsApp OTP)
MSG91_AUTH_KEY=xxxxxxxxxxxxxxxxxxxxxxxxxxxx
MSG91_TEMPLATE_ID=xxxxxxxxxxxxxxxxxxxxxxxxxxxx
MSG91_SENDER_ID=KRISHN

# App
NEXT_PUBLIC_APP_URL=http://localhost:3000
ADMIN_EMAIL=admin@babystepsindia.in

```

Create `.env.example` (same structure, empty values — commit this to git):

```

env

```



```
NEXT_PUBLIC_SUPABASE_URL=  
NEXT_PUBLIC_SUPABASE_ANON_KEY=  
SUPABASE_SERVICE_ROLE_KEY=  
RAZORPAY_KEY_ID=  
RAZORPAY_KEY_SECRET=  
RAZORPAY_WEBHOOK_SECRET=  
RESEND_API_KEY=  
RESEND_FROM_EMAIL=  
MSG91_AUTH_KEY=  
MSG91_TEMPLATE_ID=  
MSG91_SENDER_ID=  
NEXT_PUBLIC_APP_URL=  
ADMIN_EMAIL=
```

⚠ Add `.env.local` to `.gitignore`. Never commit real credentials.

3.5 Set Up Supabase Clients

typescript

```
// src/lib/supabase/client.ts  
import { createBrowserClient } from '@supabase/ssr'  
  
export function createClient() {  
  return createBrowserClient(  
    process.env.NEXT_PUBLIC_SUPABASE_URL!,  
    process.env.NEXT_PUBLIC_SUPABASE_ANON_KEY!  
  )  
}
```

typescript

```
// src/lib/supabase/server.ts
import { createServerClient } from '@supabase/ssr'
import { cookies } from 'next/headers'

export function createClient() {
  const cookieStore = cookies()
  return createServerClient(
    process.env.NEXT_PUBLIC_SUPABASE_URL!,
    process.env.NEXT_PUBLIC_SUPABASE_ANON_KEY!,
    {
      cookies: {
        get(name: string) { return cookieStore.get(name)?.value },
        set(name, value, options) { cookieStore.set({ name, value, ...options }) },
        remove(name, options) { cookieStore.set({ name, value: '', ...options }) },
      },
    }
  )
}
```

typescript

```

// src/middleware.ts
import { createServerClient } from '@supabase/ssr'
import { NextResponse, type NextRequest } from 'next/server'

export async function middleware(request: NextRequest) {
  let response = NextResponse.next({ request: { headers: request.headers } })

  const supabase = createServerClient(
    process.env.NEXT_PUBLIC_SUPABASE_URL!,
    process.env.NEXT_PUBLIC_SUPABASE_ANON_KEY!,
    {
      cookies: {
        get(name) { return request.cookies.get(name)?.value },
        set(name, value, options) {
          request.cookies.set({ name, value, ...options })
          response.cookies.set({ name, value, ...options })
        },
        remove(name, options) {
          request.cookies.set({ name, value: '', ...options })
          response.cookies.set({ name, value: '', ...options })
        },
      },
    },
  )

  const { data: { session } } = await supabase.auth.getSession()

  // Protect student routes
  const studentProtected = ['/my-courses', '/watch', '/enroll']
  const isStudentProtected = studentProtected.some(path =>
    request.nextUrl.pathname.startsWith(path)
  )
  if (isStudentProtected && !session) {
    return NextResponse.redirect(
      new URL(`/login?returnTo=${request.nextUrl.pathname}`, request.url)
    )
  }

  // Protect admin routes
  if (request.nextUrl.pathname.startsWith('/admin') &&
    !request.nextUrl.pathname.startsWith('/admin/login')) {
    if (!session) {
      return NextResponse.redirect(new URL('/admin/login', request.url))
    }
  }
}

```

```
    }  
    // Check admin flag via service role (handled in API layer)  
  }  
  
  return response  
}  
  
export const config = {  
  matcher: ['/(?!_next/static|_next/image|favicon.ico).*'],  
}
```

3.6 Generate Supabase TypeScript Types

```
bash  
  
npx supabase gen types typescript \  
  --project-id <your-supabase-project-id> \  
  --schema public \  
  > src/types/database.ts
```

Replace `<your-supabase-project-id>` with the ID from your Supabase project URL.

Phase 4 — Configure Integrations

4.1 Razorpay Setup

```
typescript
```

```
// src/lib/razorpay.ts
import Razorpay from 'razorpay'
import crypto from 'crypto'

export const razorpay = new Razorpay({
  key_id: process.env.RAZORPAY_KEY_ID!,
  key_secret: process.env.RAZORPAY_KEY_SECRET!,
})

export function verifyWebhookSignature(
  body: string,
  signature: string
): boolean {
  const expectedSignature = crypto
    .createHmac('sha256', process.env.RAZORPAY_WEBHOOK_SECRET!)
    .update(body)
    .digest('hex')
  return expectedSignature === signature
}

export async function createOrder(amountInRupees: number) {
  return razorpay.orders.create({
    amount: amountInRupees * 100, // convert to paise
    currency: 'INR',
    receipt: `receipt_${Date.now()}`,
  })
}
```

4.2 Resend Email Setup

typescript

```

// src/lib/resend.ts
import { Resend } from 'resend'

export const resend = new Resend(process.env.RESEND_API_KEY)

export async function sendEnrollmentEmail(
  to: string,
  studentName: string,
  courseTitle: string,
  intent: string
) {
  const message = intent === 'paid'
    ? `Payment confirmed! You now have full access to "${courseTitle}".`
    : `You are now enrolled in "${courseTitle}". Start learning now!`

  await resend.emails.send({
    from: process.env.RESEND_FROM_EMAIL!,
    to,
    subject: `🙏 You're enrolled in ${courseTitle} - Krishnamargam`,
    html: `
      <h2>Namaste ${studentName}!</h2>
      <p>${message}</p>
      <a href="${process.env.NEXT_PUBLIC_APP_URL}/my-courses"
        style="background:#f0b429;color:#1a0f00;padding:12px 24px;border-radius:8px;
        display:inline-block;text-decoration:none;">
        Start Learning
      </a>
      <p style="color:#666;margin-top:24px;">© 2026 Krishnamargam - BabyStepsIndia.in
    `
  })
}

export async function sendQAReplyEmail(
  to: string,
  studentName: string,
  courseTitle: string,
  lessonTitle: string,
  question: string,
  reply: string
) {
  await resend.emails.send({
    from: process.env.RESEND_FROM_EMAIL!,
    to,
    subject: `Your question on "${lessonTitle}" has been answered - Krishnamargam`,
  })
}

```

```

    html: `
      <h2>Namaste ${studentName}!</h2>
      <p>Your question on <strong>${lessonTitle}</strong> (${courseTitle}) has been
      <div style="background:#f5f5f5;padding:16px;border-radius:8px;margin:16px 0;">
        <p><strong>Your question:</strong> ${question}</p>
        <p><strong>Reply:</strong> ${reply}</p>
      </div>
      <a href="${process.env.NEXT_PUBLIC_APP_URL}/watch/${courseTitle}"
        style="background:#f0b429;color:#1a0f00;padding:12px 24px;border-radius:8px"
        View in Course
      </a>
      <p style="color:#666;margin-top:24px;">© 2026 Krishnamargam - BabyStepsIndia.in
    `
  })
}

export async function sendAdminNewQAEmail(
  courseTitle: string,
  lessonTitle: string,
  studentName: string,
  question: string
) {
  await resend.emails.send({
    from: process.env.RESEND_FROM_EMAIL!,
    to: process.env.ADMIN_EMAIL!,
    subject: `New question on "${lessonTitle}" - Krishnamargam Admin`,
    html: `
      <h2>New Question Received</h2>
      <p><strong>Course:</strong> ${courseTitle}</p>
      <p><strong>Lesson:</strong> ${lessonTitle}</p>
      <p><strong>Student:</strong> ${studentName}</p>
      <p><strong>Question:</strong> ${question}</p>
      <a href="${process.env.NEXT_PUBLIC_APP_URL}/admin/qa"
        style="background:#f0b429;color:#1a0f00;padding:12px 24px;border-radius:8px"
        Reply in Admin Panel
      </a>
    `
  })
}

```

4.3 MSG91 WhatsApp OTP Setup

typescript

```

// src/lib/msg91.ts
export async function sendWhatsAppOTP(
  mobile: string,
  isdCode: string
): Promise<{ success: boolean; message: string }> {
  const otp = Math.floor(100000 + Math.random() * 900000).toString()
  const fullMobile = `${isdCode.replace('+', '')}${mobile}`

  const response = await fetch('https://api.msg91.com/api/v5/whatsapp/whatsapp-outbo
    method: 'POST',
    headers: {
      'Content-Type': 'application/json',
      'authkey': process.env.MSG91_AUTH_KEY!,
    },
    body: JSON.stringify({
      integrated_number: process.env.MSG91_SENDER_ID,
      content_type: 'template',
      payload: {
        messaging_product: 'whatsapp',
        type: 'template',
        template: {
          name: 'krishnamargam_otp',
          language: { code: 'en' },
          components: [{
            type: 'body',
            parameters: [{ type: 'text', text: otp }]
          }]
        },
        to: fullMobile
      }
    })
  })

  if (response.ok) {
    // Store OTP in Supabase with expiry (handled in API route)
    return { success: true, message: otp } // return OTP to store server-side
  }

  return { success: false, message: 'Failed to send OTP' }
}

```


Store the OTP in a temporary Supabase table or in-memory cache (Redis if available) with a 10-minute expiry. Verify and delete on successful validation.

4.4 next.config.js

javascript

```
/** @type {import('next').NextConfig} */
const nextConfig = {
  images: {
    domains: ['img.youtube.com'],
  },
  async headers() {
    return [
      {
        // Prevent direct embedding of watch page in iframes
        source: '/watch/:path*',
        headers: [
          { key: 'X-Frame-Options', value: 'DENY' },
          { key: 'X-Content-Type-Options', value: 'nosniff' },
        ],
      },
    ],
  },
}

module.exports = nextConfig
```

Phase 5 — Seed the Database

5.1 Create Seed Script

Create `scripts/seed.ts`:

typescript

```

import { createClient } from '@supabase/supabase-js'

const supabase = createClient(
  process.env.NEXT_PUBLIC_SUPABASE_URL!,
  process.env.SUPABASE_SERVICE_ROLE_KEY! // use service role for seeding
)

async function seed() {
  console.log('🌱 Seeding database...')

  // 1. Insert paths
  const { error: pathError } = await supabase.from('paths').upsert([
    {
      slug: 'raas',
      name: 'RAAS',
      emoji: '🕉️',
      tagline_en: 'Vedic & Spiritual Learning',
      tagline_te: 'వేద మరియు ఆధ్యాత్మిక అభ్యాసం',
      description_en: 'A curated journey through the timeless wisdom of Vedic tradition',
      description_te: 'వేద సంప్రదాయం యొక్క శాశ్వత జ్ఞానం వైపు ఒక యాత్ర.',
      is_active: true,
      order_index: 1,
    },
    {
      slug: 'daas',
      name: 'DAAS',
      emoji: '👶',
      tagline_en: 'Coming Soon',
      tagline_te: 'త్వరలో వస్తోంది',
      is_active: false,
      order_index: 2,
    },
  ], { onConflict: 'slug' })

  if (pathError) { console.error('Path seed error:', pathError); return }
  console.log('✅ Paths seeded')

  // 2. Get RAAS path ID
  const { data: raasPath } = await supabase
    .from('paths').select('id').eq('slug', 'raas').single()
  const pathId = raasPath!.id

  // 3. Insert courses (paste the 12 course INSERT values here)

```

```

const courses = [
  {
    path_id: pathId,
    slug: 'bhagavad-gita-essentials',
    emoji: '📖',
    bg_color: '#1a0a00',
    title_en: 'Bhagavad Gita Essentials',
    title_te: 'భగవద్గీత తత్వం',
    description_en: 'A comprehensive study of the Bhagavad Gita covering all 18 chapters',
    instructor_en: 'Pandit Ramakrishna Sharma',
    instructor_te: 'పండిత రామకృష్ణ శర్మ',
    category: 'Bhagavad Gita',
    level: 'Beginner',
    badge: 'Popular',
    duration: '18h',
    is_free: false,
    price: 100,
    rating: 4.9,
    review_count: 1240,
    student_count: 4200,
    order_index: 0,
    is_published: true,
  },
  // ... add remaining 11 courses from DB schema seed data
]

const { error: courseError } = await supabase
  .from('courses').upsert(courses, { onConflict: 'slug' })

if (courseError) { console.error('Course seed error:', courseError); return }
console.log('✅ Courses seeded')

console.log('🎉 Database seeded successfully!')
}

seed()

```

5.2 Run the Seed Script

```
bash
```

```
npx tsx scripts/seed.ts
```

5.3 Create the Admin Account

1. Go to your Supabase project → **Authentication** → **Users**
2. Click **Add User**
3. Set email: `admin@krishnamargam.in` and a strong password
4. Click **Create User**
5. Go to **SQL Editor** and run:

```
sql

UPDATE profiles
SET is_admin = TRUE
WHERE id = (
  SELECT id FROM auth.users WHERE email = 'admin@krishnamargam.in'
);
```

6. Verify: `SELECT id, full_name, is_admin FROM profiles WHERE is_admin = TRUE;`
-

Phase 6 — Deploy to Vercel

6.1 Push to GitHub

```
bash

git init
git add .
git commit -m "feat: initial Krishnamargam platform"
git remote add origin https://github.com/yourusername/krishnamargam.git
git push -u origin main
```

6.2 Import to Vercel

1. Go to vercel.com → **Add New Project**
2. Click **Import** next to your `krishnamargam` GitHub repo
3. Framework preset: **Next.js** (auto-detected)
4. Root directory: leave as `/`
5. Build command: `next build` (default)
6. Output directory: `.next` (default)
7. **Do not click Deploy yet** — add env vars first

6.3 Add Environment Variables in Vercel

In the Vercel project setup, expand **Environment Variables** and add all variables from `.env.local`:

Variable	Value	Environment
<code>NEXT_PUBLIC_SUPABASE_URL</code>	your Supabase URL	Production, Preview, Development
<code>NEXT_PUBLIC_SUPABASE_ANON_KEY</code>	your anon key	Production, Preview, Development
<code>SUPABASE_SERVICE_ROLE_KEY</code>	your service role key	Production, Preview, Development
<code>RAZORPAY_KEY_ID</code>	your Razorpay key	Production
<code>RAZORPAY_KEY_SECRET</code>	your Razorpay secret	Production
<code>RAZORPAY_WEBHOOK_SECRET</code>	your webhook secret	Production
<code>RESEND_API_KEY</code>	your Resend key	Production, Preview
<code>RESEND_FROM_EMAIL</code>	noreply@babystepsindia.in	Production, Preview
<code>MSG91_AUTH_KEY</code>	your MSG91 key	Production
<code>MSG91_TEMPLATE_ID</code>	your template ID	Production
<code>MSG91_SENDER_ID</code>	KRISHN	Production
<code>NEXT_PUBLIC_APP_URL</code>	https://krishnamargam.babystepsindia.in	Production
<code>ADMIN_EMAIL</code>	admin@babystepsindia.in	Production

Use Razorpay **test** keys for Preview environment so you can test payments without real money.

6.4 Deploy

Click **Deploy**. Vercel will:

1. Clone your repo
2. Install dependencies
3. Run `next build`
4. Deploy to a `.vercel.app` domain

First deploy takes ~3 minutes. Watch the build log for errors.

6.5 Add Custom Domain

1. In Vercel project → **Settings** → **Domains**
2. Add: `krishnamargam.babystepsindia.in`
3. Vercel shows you a CNAME record to add to your DNS:

```
CNAME krishnamargam cname.vercel-dns.com
```

4. Add this in your domain registrar (GoDaddy / Namecheap / Cloudflare etc.)
5. DNS propagation: 5 minutes to 1 hour
6. Vercel auto-provisions SSL certificate

6.6 Update Supabase Redirect URLs

After adding your custom domain, go back to Supabase: **Authentication** → **Settings** → **Site URL**:

```
https://krishnamargam.babystepsindia.in
```

Phase 7 — Post-Deployment Verification

7.1 Checklist — Run through each item

Student Flow

- ☐ Home page loads with correct bilingual content
- ☐ EN / ௪ toggle switches all text correctly
- ☐ Explore page shows all 12 courses with filters working
- ☐ Course detail page opens (overlay on desktop, full page on mobile)

- ☐ Register flow: fill form → WhatsApp OTP received → email confirmation sent
- ☐ Login with test account: `priya@example.com` / `test1234`
- ☐ My Courses page shows enrolled courses with progress bars
- ☐ Enrollment page shows 3 options correctly
- ☐ "I cannot pay" → instant access granted → course appears in My Courses
- ☐ "I don't want to pay" → instant access granted → course appears in My Courses
- ☐ "Pay ₹100" → Razorpay modal opens → test payment completes → access granted
- ☐ Watch page loads with video → YouTube iFrame plays
- ☐ Q&A tab → submit a question → appears in list
- ☐ Notes tab → type notes → auto-saves
- ☐ Quiz tab (on courses with quiz) → submit answers → result shown

Admin Flow

- ☐ `/admin/login` → login with `admin@krishnamargam.in`
- ☐ Learning Paths page shows RAAS and DAAS
- ☐ Add new path → appears in list
- ☐ Edit path → changes saved
- ☐ Courses page shows all 12 courses
- ☐ Filter by path / level / status works
- ☐ Add new course → appears in student explore page
- ☐ Edit course → changes reflected immediately
- ☐ Q&A Hub shows questions from student
- ☐ Reply to question → student receives email notification
- ☐ Analytics page shows intent charts
- ☐ Export students → CSV downloads correctly

Security Checks

- ☐ Try to access `/watch/[slug]` without login → redirected to login
- ☐ Try to access `/watch/[slug]` without enrollment → redirected to enroll
- ☐ Check browser DevTools → no `youtube_video_id` visible in network responses
- ☐ Try `/admin/courses` without admin login → 403 returned
- ☐ Try Razorpay webhook with wrong signature → 400 returned

7.2 Test Razorpay Webhook Locally

Use the Razorpay CLI or ngrok to test webhooks during development:

```
bash
```

```
# Install ngrok
npx ngrok http 3000

# Copy the ngrok HTTPS URL and add to Razorpay webhook settings
# e.g. https://abc123.ngrok.io/api/payment/webhook

# Use Razorpay test card:
# Card: 4111 1111 1111 1111
# Expiry: any future date
# CVV: any 3 digits
# OTP: 1234 (test mode)
```

7.3 Performance Checks

Run Lighthouse in Chrome DevTools on the home page:

- Performance score: aim for > 80 on mobile
- LCP (Largest Contentful Paint): < 3 seconds
- CLS (Cumulative Layout Shift): < 0.1

If performance is low:

- Add `loading="lazy"` to images
 - Use `next/image` for all images
 - Add `export const dynamic = 'force-static'` to static pages
-

Phase 8 — Go Live (Switch to Production)

8.1 Switch Razorpay to Live Mode

1. Complete Razorpay KYC if not done
2. Go to Razorpay dashboard → switch to **Live Mode**
3. Generate live API keys
4. Update in Vercel environment variables:
 - `RAZORPAY_KEY_ID` → live key
 - `RAZORPAY_KEY_SECRET` → live secret
5. Update webhook URL in Razorpay to production URL
6. Redeploy Vercel

8.2 Final DNS Check

```
bash

# Verify your domain resolves correctly
nslookup krishnamargam.babystepsindia.in

# Verify SSL certificate
curl -I https://krishnamargam.babystepsindia.in
# Should return HTTP/2 200
```

8.3 Set Up Monitoring (Optional but recommended)

- **Vercel Analytics** — enable in Vercel project settings (free)
 - **Supabase logs** — monitor slow queries in Supabase Dashboard → Logs
 - **Razorpay alerts** — set up payment failure alerts in Razorpay dashboard
-

Ongoing Maintenance

Adding a New Course

1. Go to `/admin/courses` → Add Course
2. Fill all fields, add lessons with YouTube video IDs
3. Add quiz questions if needed
4. Toggle **Published** → students can see it immediately

Adding a New Learning Path (e.g. DAAS)

1. Go to `/admin/paths` → Edit DAAS → toggle Active to ON
2. Add courses under the DAAS path
3. DAAS card on home page automatically becomes active

Updating YouTube Videos

1. Go to `/admin/courses` → Edit Course → Edit Lesson
2. Paste the new YouTube video ID
3. Save — students see the new video immediately

Database Backups

Supabase free tier includes daily automated backups. To manually export:

Supabase Dashboard → Settings → Database → Backups → Download

Deploying Updates

```
bash

git add .
git commit -m "feat: your change description"
git push origin main
# Vercel auto-deploys on every push to main
```

Troubleshooting

Issue	Likely Cause	Fix
Build fails on Vercel	Missing env variable	Check all env vars are added in Vercel
Auth redirect loop	Wrong Site URL in Supabase	Update to production URL in Supabase Auth settings
Emails not sending	Resend domain not verified	Check DNS records in Resend dashboard
Razorpay webhook 400	Wrong webhook secret	Copy webhook secret from Razorpay dashboard
WhatsApp OTP not received	MSG91 template not approved	Check template status in MSG91 dashboard
Videos not loading	Lesson not found / token expired	Check enrollment is active; tokens expire in 1 hour
Admin can't login	is_admin not set	Run the UPDATE profiles SQL in Supabase
Supabase RLS blocking queries	Policy missing	Check RLS policies in Supabase → Authentication → Policies

Issue	Likely Cause	Fix
Telugu font not rendering	Font not loaded	Ensure Noto Sans Telugu is imported in layout.tsx